

Introduction to Python

Python is a high-level, interpreted general-purpose programming language. Celebrated for its clean, English-like syntax and emphasis on readability, it allows developers to write programs with fewer lines of code than many other languages.

```
In [6]: # Comments

# python3 --version (check Python version)
# python3 file.py (run a Python file)

# pip3 install library_name (install a library)
# pip3 install library_name==version (install a specific version of a lib)
# pip3 --upgrade library_name (upgrade a library to the latest version)
# pip3 uninstall library_name (uninstall a library)

# pip3 show library_name (show information about the library)
# pip3 list (list installed libraries)

# pip3 freeze > requirements.txt (save installed libraries to a file)
# pip3 install -r requirements.txt (install libraries from a file)
```

```
In [7]: # Importing libraries
import math
```

```
In [8]: # Hello, World!
print("Hello, World!")
```

Hello, World!

1. Variables and Data Types

```
In [9]: name = "Alice"
age = 30
height = 5.5
is_student = True

# Data types
print("Data types:")
print(type(name))
print(type(age))
print(type(height))
print(type(is_student))

# Multiple variable assignment
a, b, c = 1, 2.5, "Test"
print("\nMultiple variable assignment:")
print(a, b, c)

# Chained assignment
x = y = z = 10
print("\nChained assignment:")
print(x, y, z)
```

```
Data types:
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

```
Multiple variable assignment:
1 2.5 Test
```

```
Chained assignment:
10 10 10
```

2. Arithmetic Operators

```
In [10]: distance = 1000
time = 3

# Arithmetic operations (+, -, *, /, //, %, **)
print("Arithmetic operations:")
print("Addition (+):", distance + time)
print("Subtraction (-):", distance - time)
print("Multiplication (*):", distance * time)
print("Standard division (/):", distance / time)
print("Integer division (//):", distance // time)
print("Modulo or Remainder (%):", distance % time)
print("Power (**):", 5 ** 2)

# Formatting with round() function
print("\nFormatting with round() function:")
print("Rounding to 2 decimal places:", round(distance / time, 2))
```

```
Arithmetic operations:
Addition (+): 1003
Subtraction (-): 997
Multiplication (*): 3000
Standard division (/): 333.3333333333333
Integer division (//): 333
Modulo or Remainder (%): 1
Power (**): 25
```

```
Formatting with round() function:
Rounding to 2 decimal places: 333.33
```

3. Conditional Structure and Loops (with Enumerate)

```
In [11]: # If-Elif-Else
print("If-Elif-Else:")
num = 10
if num > 0 and num < 20:
    print("Positive and less than 20")
elif num < 0 or num > 20:
    print("Negative or greater than 20")
else:
    print("Zero")

# If with not
```

```

print("\nIf with not:")
if not num == 0:
    print("Not zero")

# While loop
print("\nWhile loop:")
x = 0
while x < 3:
    print(f"While index: {x}")
    x += 1

# While True with break
print("\nWhile True with break:")
counter = 0
while True:
    print(f"Counter: {counter}")
    counter += 1
    if counter >= 3:
        break

# For loop
print("\nFor loop:")
for i in range(3):
    print(f"For index: {i}")

# Enumerate function
print("\nEnumerate function:")
protocols = ["I2C", "SPI", "UART"]
for index, protocol in enumerate(protocols):
    print(f"Protocol n° {index}: {protocol}")

```

If-Elif-Else:

Positive and less than 20

If with not:

Not zero

While loop:

While index: 0

While index: 1

While index: 2

While True with break:

Counter: 0

Counter: 1

Counter: 2

For loop:

For index: 0

For index: 1

For index: 2

Enumerate function:

Protocol n° 0: I2C

Protocol n° 1: SPI

Protocol n° 2: UART

4. Strings and Advanced Text Formatting

String: ordered, mutable, allows duplicates, heterogeneous

```
In [12]: # String concatenation and formatting
greeting = "Hello"
name = "Alice"
message = greeting + ", " + name + "!"

print("String concatenation and formatting:")
print(message)

sat = "Meteor-M2"
model_accuracy = 0.98765
id_sensor = 4

# Formatting with str.format() and f-strings
print("\nFormatting with str.format() and f-strings:")
print("The satellite {} is in orbit.".format(sat))
print(f"Padding with zeros (03): Sensor ID {id_sensor:03}")

# Formatting with percentage and decimal places
print("\nFormatting with percentage and decimal places:")
print(f"Formatted percentage (.2%): Accuracy {model_accuracy:.2%}")
print(f"Pi constant limited to 3 decimals: {math.pi:.3f}")
```

String concatenation and formatting:
Hello, Alice!

Formatting with str.format() and f-strings:
The satellite Meteor-M2 is in orbit.
Padding with zeros (03): Sensor ID 004

Formatting with percentage and decimal places:
Formatted percentage (.2%): Accuracy 98.77%
Pi constant limited to 3 decimals: 3.142

5. Collective Data Structures

List

List: ordered, mutable, allows duplicates, heterogeneous

```
In [ ]: # List
mixed_list = [1, "two", 3.0, True]

# List elements access
print("List elements access:")
print("List:", mixed_list)
print("First element:", mixed_list[0])
print("Fourth element:", mixed_list[3])
print("Last element:", mixed_list[-1])

# List iteration
print("\nList iteration:")
for item in mixed_list:
    print(item)

# List slicing
```

```

print("\nList slicing:")
print("Sliced list (1:3):", mixed_list[1:3])
print("Sliced list (:2):", mixed_list[:2])
print("Sliced list (2:):", mixed_list[2:])

# List slicing with step
print("\nList slicing with step:")
print("Every second element:", mixed_list[::2])
print("Reversed list:", mixed_list[::-1])

# List concatenation
print("\nList concatenation:")
print("Concatenated list:", mixed_list + [5, "six"])

# List properties and methods
print("\nList properties and methods:")
print("Length of the list:", len(mixed_list))

# Membership test
print("\nMembership test:")
print("Membership test ('two' in mixed_list):", "two" in mixed_list)
print("Membership test ('three' not in mixed_list):", "three" not in mixed_list)

# Membership test with numbers
print("\nMembership test with numbers:")
nums = [1, 2, 3, 4, 5]
if 10 not in nums:
    print("10 is not in the list")

# Modifying lists
print("\nModifying lists:")

mixed_list[1] = "changed"
print("Modified list:", mixed_list)

mixed_list.append("new")
print("Appended element:", mixed_list)

mixed_list.insert(2, "inserted")
print("Inserted element:", mixed_list)

mixed_list.extend([7, "eight"])
print("Extended list:", mixed_list)

mixed_list.remove("changed")
print("Removed element:", mixed_list)

last_element = mixed_list.pop()
print("Popped element:", last_element)
print("List after popping:", mixed_list)

del mixed_list[0]
print("List after deleting first element:", mixed_list)
mixed_list.clear()
print("Cleared list:", mixed_list)

# Sorting lists
print("\nSorting lists:")
nums = [5, 2, 9, 1]
nums.sort()

```

```
print("Sorted list:", nums)
nums.sort(reverse=True)
print("Sorted in reverse:", nums)

# List comprehensions
print("\nList comprehensions:")

squares = [x**2 for x in range(5)]
print("List comprehension (Squares):", squares)

sats_database = ["NOAA 15", "NOAA 18", "NOAA 20", "NOAA 21"]
active_sats = [sat for sat in sats_database if "20" in sat or "21" in sat]
print("List comprehension (Filtered Satellites):", active_sats)
```

```
List elements access:  
List: [1, 'two', 3.0, True]  
First element: 1  
Fourth element: True  
Last element: True
```

```
List iteration:  
1  
two  
3.0  
True
```

```
List slicing:  
Sliced list (1:3): ['two', 3.0]  
Sliced list (:2): [1, 'two']  
Sliced list (2:): [3.0, True]
```

```
List slicing with step:  
Every second element: [1, 3.0]  
Reversed list: [True, 3.0, 'two', 1]
```

```
List concatenation:  
Concatenated list: [1, 'two', 3.0, True, 5, 'six']
```

```
List properties and methods:  
Length of the list: 4
```

```
Membership test:  
Membership test ('two' in mixed_list): True  
Membership test ('three' not in mixed_list): True
```

```
Membership test with numbers:  
10 is not in the list
```

```
Modifying lists:  
Modified list: [1, 'changed', 3.0, True]  
Appended element: [1, 'changed', 3.0, True, 'new']  
Inserted element: [1, 'changed', 'inserted', 3.0, True, 'new']  
Extended list: [1, 'changed', 'inserted', 3.0, True, 'new', 7, 'eight']  
Removed element: [1, 'inserted', 3.0, True, 'new', 7, 'eight']  
Popped element: eight  
List after popping: [1, 'inserted', 3.0, True, 'new', 7]  
List after deleting first element: ['inserted', 3.0, True, 'new', 7]  
Cleared list: []
```

```
Sorting lists:  
Sorted list: [1, 2, 5, 9]  
Sorted in reverse: [9, 5, 2, 1]
```

```
List comprehensions:  
List Comprehension (Squares): [0, 1, 4, 9, 16]  
List Comprehension (Filtered Satellites): ['NOAA 20', 'NOAA 21']
```

Set

Set: unordered, mutable, no duplicates, heterogeneous

```
In [ ]: # Set  
num_set = {1, 2, 2, 3, 4, 5}
```

```
# Set elements access
print("Set elements access:")
print("Set:", num_set)

# Set iteration
print("\nSet iteration:")
for item in num_set:
    print(item)

# Set properties and methods
print("\nSet properties and methods:")
print("Length of set:", len(num_set))

# Modifying sets
print("\nModifying sets:")

num_set.add(6)
print("Set after adding 6:", num_set)

num_set.remove(3)
print("Set after removing 3:", num_set)

num_set.discard(10)
print("Set after discarding 10:", num_set)

A = {1, 2, 3}
B = {3, 4, 5}

# Membership test
print("\nMembership test:")
print("3 in A:", 3 in A)
print("10 not in A:", 10 not in A)

# Set operations
print("\nSet operations:")

print("A | B:", A | B)
print("A union B:", A.union(B))

print("A & B:", A & B)
print("A intersection B:", A.intersection(B))

print("A - B:", A - B)
print("A difference B:", A.difference(B))

print("A ^ B:", A ^ B)
print("A symmetric difference B:", A.symmetric_difference(B))
```


Set elements access:
Set: {1, 2, 3, 4, 5}

Set iteration:

1
2
3
4
5

Set properties and methods:
Length of set: 5

Modifying sets:

Set after adding 6: {1, 2, 3, 4, 5, 6}
Set after removing 3: {1, 2, 4, 5, 6}
Set after discarding 10: {1, 2, 4, 5, 6}

Membership test:

3 in A: True
10 not in A: True

Set operations:

A | B: {1, 2, 3, 4, 5}
A union B: {1, 2, 3, 4, 5}
A & B: {3}
A intersection B: {3}
A - B: {1, 2}
A difference B: {1, 2}
A ^ B: {1, 2, 4, 5}
A symmetric difference B: {1, 2, 4, 5}

Dictionary

Dictionary: unordered, mutable, unique keys, heterogeneous values

```
In [ ]: # Dictionary
person = {
    "name": "Alice",
    "age": 30,
    "city": "New York"
}

# Alternative way to create a dictionary
person = dict(name="Alice", age=30, city="New York")

# Dictionary elements access
print("Dictionary elements access:")
print("Dictionary:", person)
print("Name:", person["name"])
print("Age:", person["age"])
print("City:", person["city"])

# Dictionary access with get() method
print("\nDictionary access with get() method:")
print("Age (using get):", person.get("age"))
print("Country (using get):", person.get("country", "Unknown"))

# Dictionary iteration
```

```
print("\nDictionary iteration:")

print("Iterating through keys:")
for key in person:
    print("Key:", key)

print("Iterating through values:")
for value in person.values():
    print("Value:", value)

print("Iterating through key-value pairs:")
for key, value in person.items():
    print(f"{key}: {value}")

# Methods and properties
print("\nMethods and properties:")
print("Number of key-value pairs:", len(person))

# Membership test
print("\nMembership test:")
print("name" in person)
print("city" not in person)

# Dictionary views
print("\nDictionary views:")
print("Keys:", person.keys())
print("Values:", person.values())
print("Items:", person.items())

# Modifying dictionaries
print("\nModifying dictionaries:")

person["age"] = 31
print("Updated Dictionary:", person)

person["country"] = "USA"
print("Added Key-Value Pair:", person)

person.pop("age")
print("After Popping 'age':", person)

person.popitem()
print("After Popping Last Item:", person)

del person["city"]
print("After Deleting 'city':", person)

person.clear()
print("After Clearing Dictionary:", person)

# Nested dictionaries
print("\nNested dictionaries:")
students = {
    "Alice": {"age": 30, "grade": "A"},
    "Bob": {"age": 25, "grade": "B"}
}

print("Nested Dictionary:", students)
print("Alice's grade:", students["Alice"]["grade"])
print("Bob's age:", students["Bob"]["age"])
```

Dictionary elements access:

Dictionary: {'name': 'Alice', 'age': 30, 'city': 'New York'}

Name: Alice

Age: 30

City: New York

Dictionary access with get() method:

Age (using get): 30

Country (using get): Unknown

Dictionary iteration:

Iterating through keys:

Key: name

Key: age

Key: city

Iterating through values:

Value: Alice

Value: 30

Value: New York

Iterating through key-value pairs:

name: Alice

age: 30

city: New York

Methods and properties:

Number of key-value pairs: 3

Membership test:

True

False

Dictionary views:

Keys: dict_keys(['name', 'age', 'city'])

Values: dict_values(['Alice', 30, 'New York'])

Items: dict_items([('name', 'Alice'), ('age', 30), ('city', 'New York')])

Modifying dictionaries:

Updated Dictionary: {'name': 'Alice', 'age': 31, 'city': 'New York'}

Added Key-Value Pair: {'name': 'Alice', 'age': 31, 'city': 'New York', 'country': 'USA'}

After Popping 'age': {'name': 'Alice', 'city': 'New York', 'country': 'USA'}

After Popping Last Item: {'name': 'Alice', 'city': 'New York'}

After Deleting 'city': {'name': 'Alice'}

After Clearing Dictionary: {}

Nested dictionaries:

Nested Dictionary: {'Alice': {'age': 30, 'grade': 'A'}, 'Bob': {'age': 25, 'grade': 'B'}}

Alice's grade: A

Bob's age: 25

Tuple

Tuple: ordered, immutable, allows duplicates, heterogeneous

```
In [ ]: # Tuple
        point = (1, 2)
```

```
# Tuple elements access
print("Tuple elements access:")
print("Tuple:", point)
print("First element:", point[0])
print("Second element:", point[1])
```

Tuple elements access:

Tuple: (1, 2)

First element: 1

Second element: 2

6. Functions and Recursion

```
In [22]: # Function
def sum(a, b):
    return a + b

print("Function:")
print(sum(3, 5))

# Default parameter values
def greet(name="World"):
    return f"Hello, {name}!"

print("\nDefault parameter values:")
print(greet())
print(greet("Alice"))

# Global vs Local variables
x = 10 # Global variable
def modify_global():
    global x # Declare x as global to modify it
    y = 5 # Local variable
    x += y # Modify global variable x

print("\nGlobal vs Local variables:")
modify_global()
print("Modified global variable x:", x)

# Nested functions
def outer_function():
    def inner_function():
        return "Hello from the inner function!"
    return inner_function()

print("\nNested functions:")
print(outer_function())

# Recursion (factorial)
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

print("\nRecursion (factorial):")
print("Factorial of 5:", factorial(5))

# Recursion (Fibonacci)
```

```
def fibonacci(n):  
    if n <= 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)  
  
print("\nRecursion (Fibonacci):")  
print("Fibonacci of 10:", fibonacci(10))
```

Function:

8

Default parameter values:

Hello, World!

Hello, Alice!

Global vs Local variables:

Modified global variable x: 15

Nested functions:

Hello from the inner function!

Recursion (factorial):

Factorial of 5: 120

Recursion (Fibonacci):

Fibonacci of 10: 55